

Distributed Search Engine

Ever wondered what happens behind the scenes when you type a query into Google and get relevant results in milliseconds? Now it's your turn to find out. In this project, you will build your own **mini-Google** — a miniature search engine that transforms a corpus of documents into a searchable, intelligently ranked knowledge base, with the heavy computation spread across four connected VMs.

1. Overview

Search is the problem that produced much of today's big data tooling: GFS, MapReduce and PageRank all came out of it. This project asks you to build a small version end to end, and to observe where the time and network traffic actually go. You will work with distributed storage, parallel computation, and versioned data — and discover what happens when data changes or a component fails. Your system will consist of:

- **One Ingestion Node**: The gateway that brings new documents into your search engine.
- **Two Cluster Nodes**: The backbone of your system, storing data and powering distributed indexing and ranking.
- **One Search Node**: Delivers ranked search results based on query.

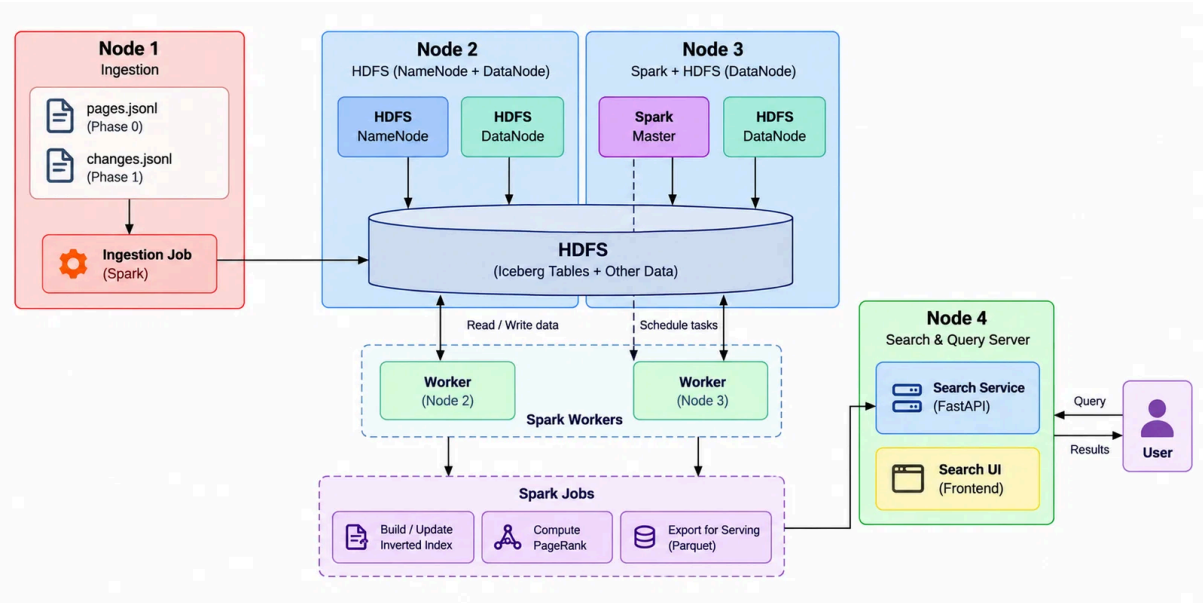
2. Objective

Design and implement a distributed search engine that demonstrates:

Made with Super

- **Seamless data ingestion:** Handles duplicates, malformed records, and updates reliably.
- **Fast retrieval:** Build a distributed inverted index to find relevant documents for keyword queries.
- **Intelligent ranking:** Combine **BM25** text relevance with **PageRank** to deliver meaningful search results.
- **Versioned, resilient data:** Use Apache Iceberg snapshots to track changes, travel back to earlier versions, and recover from bad updates.
- **Ranked query serving:** Serve ranked search results through an API, powered by a distributed HDFS and Spark cluster.

3. System Architecture



Node 1 — Ingestion

- **Owns ingestion and data quality.** Holds the provided corpus (see Dataset below) on local disk and uploads to HDFS.
- Writes the ingestion Spark job to parse and validate input, reject malformed records or records missing required fields, deduplicate by document ID (highest version wins), remove links to documents that do not exist, and merge accepted changes into Iceberg tables.
- Detects changes from Iceberg so later phases can be processed consistently; ingesting the same phase twice should not change the resulting table contents.
- Runs the Spark driver for ingestion. This node is a client and does not run a Spark or HDFS cluster daemon. The job is submitted to Spark master (Node 3).

Node 2 — HDFS Namenode, Datanode + Spark worker

Made with Super

- **Owns storage and the lakehouse.** Sets up the HDFS NameNode and DataNode; HDFS uses replication factor 2 across Nodes 2 and 3.
- Runs a Spark worker and maintains the Iceberg tables.
- Stores the Iceberg table data and metadata in HDFS.

Node 3 — Spark Master, worker + HDFS Datanode

- **Owns distributed compute.** Runs the Spark master, a Spark worker, and an HDFS DataNode; this machine also participates in distributed storage.
- Hosts the Spark drivers for indexing, PageRank, and export. The workers on Nodes 2 and 3 execute the distributed tasks.

Node 4 — Search Service + UI

- Hosts the FastAPI search service and loads the validated, versioned Parquet export into memory.
- Calculates **BM25** (see Resources) and combines it with precomputed PageRank.
- Provides a basic search UI/frontend that allows users to enter keyword queries and view ranked search results.
- Serves query results with relevant document information (e.g., document ID and URL), without running Spark/HDFS daemons or querying Iceberg directly.

4. Dataset

You are provided with the following datasets **(**download here**)**:

- **Phase 0 —** `pages.jsonl` **:** The base corpus containing around 1,000 pages and 500 links from Simple English Wikipedia.
- **Phase 1 —** `changes.jsonl` **:** A change set to apply on top of Phase 0, used to test how updates are reflected in the versioned tables and search results.

Running the full pipeline takes approximately **10–15 minutes per phase** on the provided corpus. You may use this toy dataset for faster experimentation and debugging, but the final project must be executed on the main corpus only.

End-to-end workflow

1. **Phase 0:** Start with `pages.jsonl` . Node 1 uploads this input file to HDFS and runs the ingestion job.
2. The ingestion job validates records and merges accepted changes into Iceberg tables on HDFS.
3. Node 2 hosts the HDFS namenode and one datanode, while Node 3 runs [illegible]nd datanode. Together, the datanodes store the replicated HDFS

Made with Super

blocks (**replication factor = 2**).

4. Spark jobs are submitted to the Spark master on Node 3 to build/update the inverted index and compute PageRank (minimum **10** iterations) across the workers on Nodes 2 and 3.
5. Node 3 runs the Spark export job to write the validated index and document metadata as plain Parquet files for serving.
6. Node 4 pulls the completed export into its local serving directory (for example, `/serving/vN/`).
7. The search service on Node 4 switches to a new version only if its export finished successfully.
8. **Phase 1:** After Phase 0 has completed and its results are observed, upload `changes.jsonl` to HDFS and rerun the same pipeline stages: ingestion, validation, inverted index update, PageRank, and export to Node 4. Keep the Phase 0 search version available if validation or export fails.

5. Resources

- **Apache Iceberg** hands-on guide
- **BM25** (Best Matching 25) **:** An industry-standard ranking algorithm that improves on traditional TF-IDF by adding term frequency saturation (repeating a word multiple times yields diminishing returns) and document length normalization (preventing longer documents from gaining an unfair advantage).

  For each query term t present in document D, BM25 is calculated as:

$$\text{BM25}(D, Q) = \sum_{t \in Q} \text{IDF}(t) \times \frac{\text{TF}(t, D) \times (k_1 + 1)}{\text{TF}(t, D) + k_1 \times \left(1 - b + b \times \frac{\text{len}(D)}{\text{avgdl}}\right)}$$

You may use this code snippet for the project.

Ranking Configuration

For this project, use BM25 parameters k_1 **= 1.2** and **b = 0.75**, with a PageRank damping factor of **0.85**.

Combine normalized scores using:

$$\text{Final Score} = 0.7 \times \left(\frac{\text{BM25}}{\max \text{BM25}}\right) + 0.3 \times \left(\frac{\text{PageRank}}{\max \text{PageRank}}\right)$$

6. Tech Stack

Made with Super

Technology	Role
ZeroTier	Private network connecting the four VMs
HDFS	Distributed storage across Nodes 2 and 3
Apache Spark	Distributed ingestion, indexing, PageRank and exp
Apache Iceberg	Table format on HDFS; supports updates/merges snapshots
FastAPI + any frontend tool/framework	Search service + UI

7. Expected Deliverables

1. **Phase-based ingestion and lakehouse tables**
   - Ingestion of the provided base corpus and the subsequent change set.
   - Validation and handling of dirty or inconsistent records.
2. **Distributed inverted index**
3. **PageRank computation (minimum 10 iterations)**
4. **Spark fault tolerance**
5. **BM25 relevance ranking**
6. **Search service**
   - A usable search endpoint/service on Node 4 along with a **basic frontend**.
   - Query responses containing ranked documents and useful identifying information (for example, document ID and URL).
   - Demonstrate queries against the initial corpus and after changes are applied.

Evaluation Rubrics

Teams must arrive **with Phase 0 already executed and serving**. The search service on VM4 must already be running, hosting the Phase 0 Parquet export in memory. Only Phase 1 will be triggered live during evaluation.

No.	Description	Marks
	Infrastructure & Phase 0	**3 M**

Made with Super

	Verification	
2	Live Phase 1 Demo	6 M
3	Search Verification	3 M
4	Idempotency Test	2 M
5	Individual Viva	6 M
	Grand Total	20 M (reduced to 10 M)

Made with Super